

```
class Banner
```

```
attr_accessible :horiz, :link, :visible, :image, :position  
has_attached_file :image, styles: { vert: '220'
```

```
before_create :assign_position
```

```
class Banner < ActiveRecord::Base
```

```
attr_accessible :horiz, :link, :visible, :image  
has_attached_file :image, styles: { vert: '220'
```

```
before_create :assign_position
```

```
protected
```

```
def assign_position
```

```
max = Banner.maximum(:position)
```

```
self.position = max ? max + 1 : 0
```

FULL STACK DEVELOPMENT

REACT – O PRINCÍPIO

AULA 06

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS	14

EMANSP

O QUE VEM POR AÍ?

Nesta aula, exploraremos como conectar nossos apps React a outras partes da internet de forma eficiente e segura. Vamos abordar também técnicas essenciais para tratamento de erros. Isso garantirá que nossos usuários tenham uma experiência agradável enquanto nossos dados permanecem protegidos.



HANDS ON

Neste vídeo, abordaremos um tópico essencial para o desenvolvimento de aplicações modernas: conectar nossos apps React com APIs externas utilizando as técnicas e ferramentas mais atuais. Detalharemos como utilizar fetch e axios para realizar essas conexões de maneira eficaz, além de explorar métodos práticos para manipular os dados recebidos, como filtrar, ordenar e combinar informações.



SAIBA MAIS

Conectando React com APIs – O que são APIs?

APIs, ou Interfaces de Programação de Aplicações, são fundamentais para o desenvolvimento moderno de software. Elas permitem que diferentes sistemas e aplicações compartilhem dados e funcionalidades de maneira segura, eficiente e controlada. Em termos simples, uma API define um contrato de como os componentes de um software interagem entre si, com especificações claras para a troca de informações. Isso significa que uma aplicação pode acessar os recursos de outra aplicação sem precisar entender como todo o sistema é implementado.

No contexto da web, APIs geralmente se referem a web services ou REST APIs, que são acessadas via HTTP. Essas APIs são projetadas para serem consumidas por aplicações web, móveis ou de desktop, e são fundamentais para a construção de serviços interconectados.

Para explicar a importância e o papel das APIs no desenvolvimento moderno de software, vamos criar um contexto hipotético em que o conceito de API é crucial: você faz parte de uma equipe de desenvolvimento encarregada de criar um aplicativo móvel de previsão do tempo. O aplicativo deve ser capaz de mostrar a previsão do tempo atual, previsões horárias e previsões para os próximos dias. Além disso, ele deve oferecer funcionalidades como alertas de clima severo e recomendações de vestimenta baseadas nas condições climáticas.



Figura 1 – APIs
Fonte: gerada por IA (2024)

Solução através das APIs

Para fornecer informações meteorológicas precisas e atualizadas, seu aplicativo precisa acessar dados de clima em tempo real. No entanto, coletar e processar dados meteorológicos globalmente é complexo e requer infraestrutura especializada e acesso a uma variedade de sensores e estações meteorológicas. Nesse momento as APIs se tornam indispensáveis.

Você decide integrar seu aplicativo com uma API de previsão do tempo de terceiros, como a OpenWeatherMap ou a AccuWeather API. Essas APIs web oferecem acesso programático a vastos conjuntos de dados meteorológicos, incluindo temperaturas, precipitação, vento e muito mais.

Implementando a Integração com a API

Seleção da API: após uma pesquisa, sua equipe seleciona uma API que melhor atende às necessidades do aplicativo em termos de variedade de dados, confiabilidade e termos de uso.

Autenticação e Autorização: para usar a API, você se registra na plataforma fornecedora e obtém uma chave de API, que é utilizada para autenticar suas requisições.

Desenvolvimento e testes: utilizando as documentações da API, sua equipe desenvolve funcionalidades que fazem requisições HTTP para a API, solicitando dados de previsão do tempo para localidades especificadas pelo usuário. Você implementa tratamento de respostas e erros, garantindo que o aplicativo possa lidar com diferentes cenários, como falhas de rede ou dados indisponíveis.

Lançamento e monitoramento: com a integração completa, você lança o aplicativo. Após o lançamento, monitora o uso da API para garantir que as requisições permaneçam dentro dos limites permitidos e ajusta a implementação conforme necessário para otimizar o desempenho e a experiência do usuário.

Benefícios das APIs no projeto

Eficiência e economia de tempo: ao utilizar uma API externa, sua equipe pode se concentrar no desenvolvimento das funcionalidades do aplicativo, sem a necessidade de construir uma infraestrutura complexa de coleta e processamento de dados meteorológicos.

Acesso a dados atualizados e confiáveis: as APIs fornecem acesso a dados em tempo real, mantidos por especialistas e atualizados continuamente, garantindo que o aplicativo forneça informações precisas aos usuários.

Flexibilidade: integrar com uma API permite que seu aplicativo se adapte facilmente a mudanças ou melhorias nos dados meteorológicos, sem a necessidade de grandes revisões no código do aplicativo.

Conectando com APIs usando Fetch

O método `fetch` é uma maneira nativa de realizar requisições HTTP em JavaScript, tornando-o uma escolha popular para aplicações React quando se trata de consumir APIs. Sua simplicidade e a promessa de suporte nativo em navegadores modernos o tornam uma ferramenta poderosa.

Para usar `fetch`, você passa a URL da API como argumento, opcionalmente acompanhado de um objeto de opções para configurar a requisição, e o `fetch` retorna uma promessa que, quando resolvida, fornece um objeto de resposta que você pode usar para acessar os dados retornados pela API. Aqui está um exemplo básico de uso do `fetch`:

```
fetch('https://api.example.com/data')
  .then(response => {
    if (!response.ok) {
      throw new Error('Network response was not ok');
    }
    return response.json();
  })
  .then(data => console.log(data))
  .catch(error => console.error('There has been a problem with your fetch operation:', error));
```

Utilizando Axios para Requisições

Apesar do `fetch` ser poderoso e adequado para muitos casos de uso, algumas aplicações requerem funcionalidades adicionais ou uma sintaxe mais simplificada. Aqui é onde `axios` brilha. `Axios` é uma biblioteca cliente HTTP baseada em promessas, que funciona tanto no navegador quanto em `node.js`, e oferece uma API mais rica para fazer requisições.

Para começar com `axios`, primeiro você precisa instalá-lo:

```
npm install axios
```

Em seguida, você pode realizar uma requisição da seguinte maneira:


```
import axios from 'axios';

axios.get('https://api.example.com/data')
  .then(response => console.log(response.data))
  .catch(error => console.error('Error:', error));
```

Um dos principais benefícios de usar axios é que ele converte automaticamente a resposta JSON, eliminando a necessidade de chamar `.json()` na resposta, como você faria com `fetch`. Além disso, axios tem um tratamento de erro mais intuitivo e suporte incorporado para o progresso de upload e download.

Tratamento de erros

O tratamento de erros é um aspecto crítico quando se trata de interagir com APIs. Tanto `fetch` quanto `axios` oferecem maneiras de capturar e tratar erros que podem ocorrer durante a requisição. É importante não apenas capturar esses erros, mas também informar ao usuário de maneira adequada e tomar as devidas providências para uma boa experiência do usuário.

Com `fetch`, um erro é lançado apenas em caso de falha de rede ou se algo impedir a requisição de ser feita. Não é lançado um erro para respostas HTTP indicando erro (como 404 ou 500). Portanto, você precisa verificar explicitamente a propriedade `ok` do objeto de resposta.

Por outro lado, `axios` lança um erro quando a requisição falha por motivos de rede ou se o status da resposta está fora do intervalo 2xx. Isso pode simplificar o código de tratamento de erro, permitindo que você coloque toda a lógica de erro dentro do bloco `catch`.

Exemplos comuns de erros ao usar fetch

Erros de rede: quando a requisição não pode ser feita devido a problemas de conexão. Isso pode ser capturado verificando se a promessa foi rejeitada.

```

fetch('https://api.example.com/data')
  .then(response => {
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }
    return response.json();
  })
  .catch(error => console.log('Fetch error: ' + error.message));

```

Erros de timeout: fetch não suporta nativamente o timeout de requisições, mas isso pode ser implementado usando Promise.race com um temporizador customizado.

```

const fetchWithTimeout = (url, options, timeout = 5000) => {
  return Promise.race([
    fetch(url, options),
    new Promise( (_, reject) =>
      setTimeout(() => reject(new Error('Request timed out')), timeout)
    )
  ]);
};

```

Erros de parsing: ao tentar converter a resposta para JSON quando o servidor retorna algo que não é JSON válido.

```

fetch('https://api.example.com/data')
  .then(response => response.json())
  .catch(error => console.log('Parsing error: ' + error.message));

```

Exemplos comuns de erros ao usar axios

Erro de resposta: quando o servidor responde com um status fora do intervalo 2xx, axios automaticamente lança um erro que pode ser capturado no bloco catch.

```

axios.get('https://api.example.com/data')
  .catch(error => {
    if (error.response) {
      // O servidor respondeu com um status fora do intervalo 2xx
    }
  });

```

```

    console.log(`Server responded with status: ${error.response.status}`);
  } else if (error.request) {
    // A requisição foi feita mas não houve resposta
    console.log('No response from server');
  } else {
    // Algo aconteceu na configuração da requisição
    console.log('Error', error.message);
  }
});

```

Erro de cancelamento: axios suporta o cancelamento de requisições. Isso pode ser útil em situações como a desmontagem de um componente React antes da conclusão da requisição.

```

const CancelToken = axios.CancelToken;
const source = CancelToken.source();

axios.get('https://api.example.com/data', { cancelToken: source.token })
  .catch(function (thrown) {
    if (axios.isCancel(thrown)) {
      console.log('Request canceled', thrown.message);
    } else {
      // handle error
    }
  });

// Cancela a requisição
source.cancel('Operation canceled by the user.');
```

Erros de configuração: ocorrem quando a configuração da requisição é inválida. Por exemplo, passar parâmetros errados para o cabeçalho da requisição.

```

axios.get('https://api.example.com/data', { headers: { 'Invalid-Header': 'value' } })
  .catch(error => console.log('Configuration error: ' + error.message));

```

Implementando feedback visual para o usuário

Além de tratar erros, é uma prática recomendada implementar um feedback visual para o usuário em casos de carregamento e erro, garantindo que a aplicação

permaneça responsiva e informativa, mesmo quando as coisas não saem como planejado. Veja alguns exemplos:

- Exibir mensagens de erro claras e específicas para o usuário, evitando detalhes técnicos desnecessários;
- Mostrar ícones ou animações de carregamento durante a realização de requisições;
- Implementar telas de erro ou modais que informem o usuário sobre problemas ao carregar dados e ofereçam ações de recuperação, como tentar novamente.

Lidar com erros de forma eficaz e transparente para o usuário não apenas melhora a experiência, mas também contribui para a confiança na sua aplicação.

Logo, conectar sua aplicação React a APIs externas usando fetch ou axios é um passo crucial para criar aplicações web modernas e interativas. Ambos os métodos têm seus próprios conjuntos de vantagens e peculiaridades, permitindo que desenvolvedores(as) escolham a ferramenta mais adequada para suas necessidades específicas. Além disso, um tratamento de erro eficaz é essencial para garantir uma experiência de usuário suave e profissional. Com estas ferramentas e técnicas, você tem o conhecimento necessário para fazer o melhor uso das APIs no seu próximo projeto React.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, mergulhamos no mundo das APIs, explorando sua importância fundamental no desenvolvimento de software moderno e como elas permitem a interação segura e eficiente entre diferentes sistemas e aplicações. Aprofundamos no uso das funções `fetch` e `axios` para conectar aplicações React a APIs externas, destacando as peculiaridades de cada uma, incluindo a manipulação de respostas e a importância crítica do tratamento de erros.

Vimos exemplos comuns de erros que podem surgir ao fazer requisições, como erros de rede, parsing e timeout com `fetch`, além de erros de resposta e cancelamento com `axios`. Discutimos estratégias para lidar efetivamente com esses erros, enfatizando a necessidade de fornecer feedback visual ao usuário para uma experiência aprimorada. Esta aula forneceu as ferramentas e conhecimentos necessários para construir aplicações web robustas e interativas, capazes de consumir e interagir com dados de APIs de forma eficiente e segura.

REFERÊNCIAS

ALURA. **React Native: fazendo requisições com Fetch**. 2021. Disponível em: <https://www.alura.com.br/artigos/react-native-fazendo-requisicoes-fetch>. Acesso em: 01 jul. 2024.

ALURA. **Requisições HTTP utilizando o AXIOS**. 2022. Disponível em: <https://www.alura.com.br/artigos/requisicoes-http-utilizando-axios>. Acesso em: 01 jul. 2024.

REACT. **Documentação React**. [s.d.]. Disponível em: <https://react.dev/>. Acesso em: 01 jul. 2024.

PALAVRAS-CHAVE

Palavras-chave: ReactJs. Front-end. Biblioteca. React-hooks.

EMSE

POS TECH